



Análisis Exploratorio de Datos (EDA)

Conjunto de técnicas estadísticas dirigidas a explorar, describir y resumir la información que contienen los datos, maximizando su comprensión

Desarrolladas originalmente por el matemático John Tukey en la década de 1970

Su objetivo no es confirmar hipótesis sino generar preguntas y posibles direcciones para las investigaciones futuras

EDA en CDD es el arte de hacer preguntas más que el de buscar respuestas específicas

Requiere una actitud de curiosidad y apertura mental, tratando de explorar los datos con una mente abierta, sin hipótesis preconcebidas

Al practicar EDA no solo descubrimos lo que buscamos, sino que a menudo nos encontramos con sorpresas que nos llevan por caminos inesperados

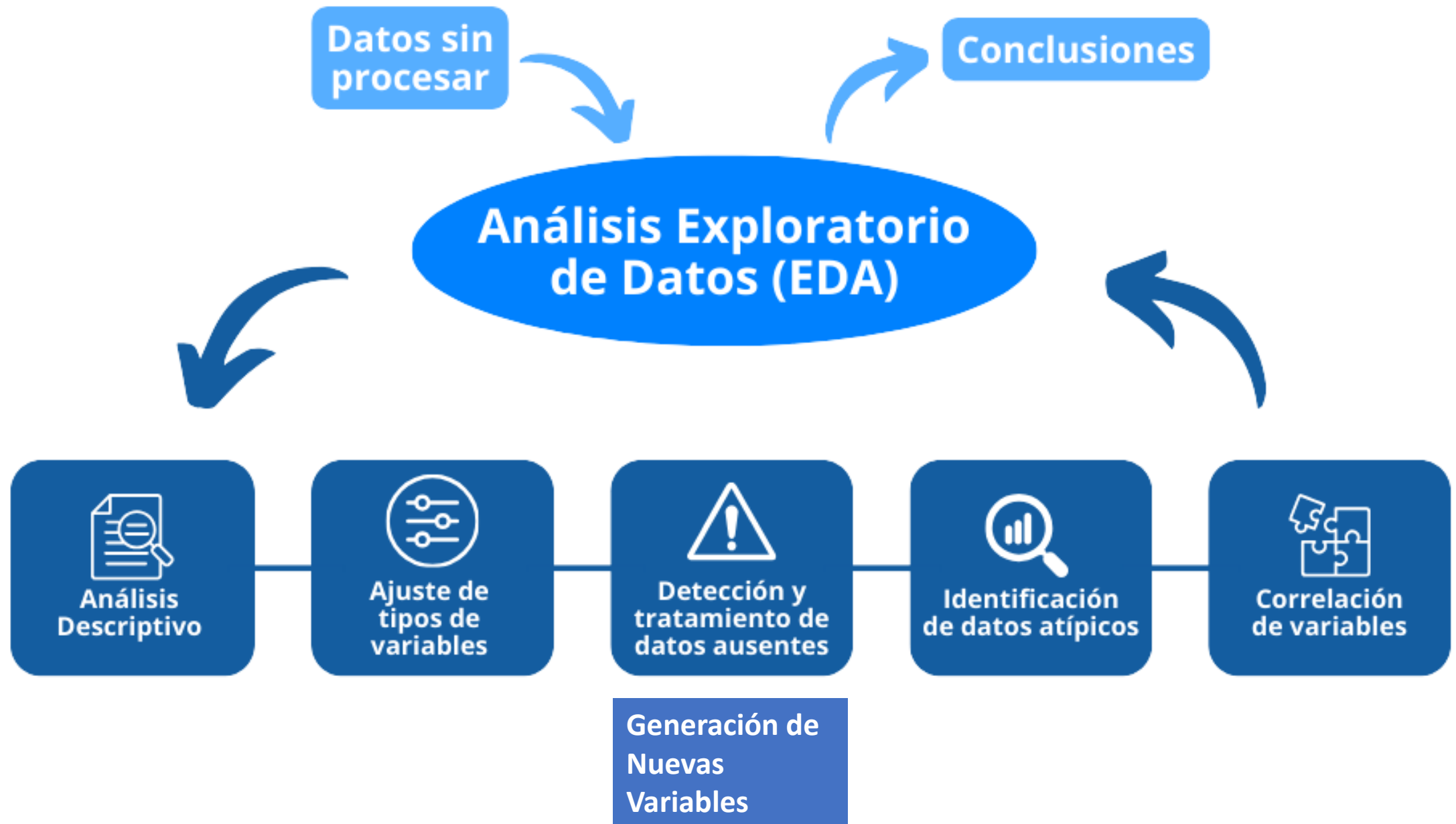
¿Por qué hacer EDA?

EDA es esencial para garantizar que los resultados de cualquier análisis estadístico sean consistentes y veraces

¿Qué Implica?

- ✓ Realizar un análisis descriptivo de los datos
- ✓ Identificar la presencia de posibles errores u omisiones
- ✓ Revelar la presencia de datos atípicos
- ✓ Comprobar la relación entre diversas variables
- ✓ Generar nuevas variables





Análisis Descriptivo:

¿Qué es?

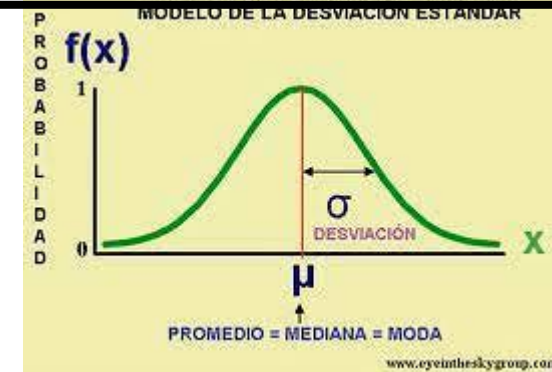
Síntesis de la información que proporciona el conjunto de datos, extrayendo sus características más representativas

¿Por qué es necesario?

Permite conocer los tipos de datos, descubrir patrones y preparar los datos para futuros análisis

Tratamiento

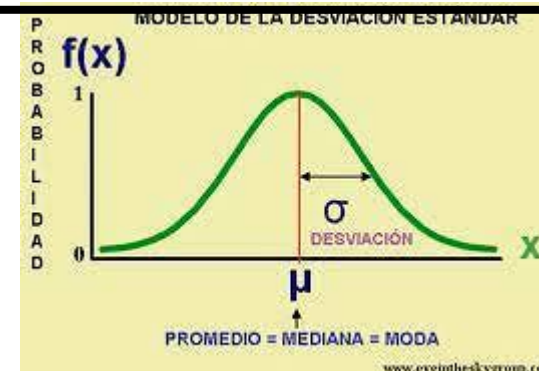
Aplicar funciones de estadística descriptiva para explorar la estructura del conjunto de datos, examinar los datos y las variables que presenta



Análisis Descriptivo:

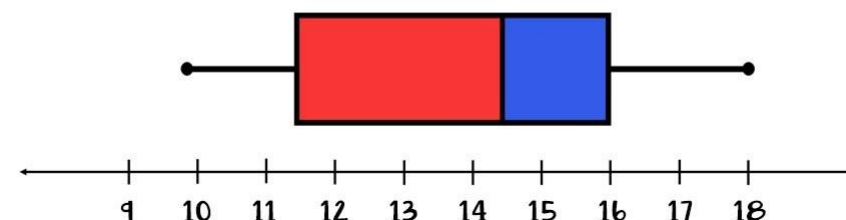
¿Qué averiguamos sobre los datos?

Tipo de función, mínimos y máximos, medidas de tendencia central, medidas de dispersión, rango intercuartílico



CAJAS Y BIGOTES BOXPLOT

11 15 14 16 17 10 15
11 18 16 12 14 15 14



Ajuste de Tipos de Variables (Ingeniería de Atributos):

¿Qué es?

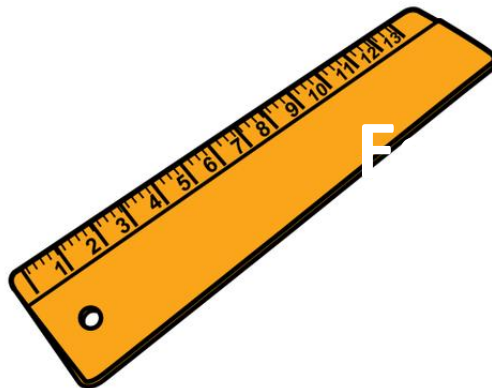
Verificar que las variables se han almacenado con el tipo de valor adecuado

¿Por qué es necesario?

Una mala codificación de las variables puede influir negativamente en la agrupación de los datos o los resultados de los análisis

Tratamiento

Aplicar la codificación apropiada para cada una de las variables





Tipos de variables

Categóricas o cualitativas: No se puede establecer un ordenamiento entre ellas

Binarias: Toman uno de dos valores disjuntos y complementarios

Cuasicuantitativas u ordinales: Si bien se puede ordenar las modalidades que adopta, no se puede establecer una distancia entre ellas

Cuantitativas discretas: Toman valores numéricos siendo que entre dos valores consecutivos de las mismas no existen valores intermedios

Cuantitativas continuas: También toman valores numéricos, pero entre dos valores de la variable existen infinitos valores intermedios



Tipos de variables

Algunos algoritmos de ML no aceptan variables categóricas
Si son pocas categorías se pueden generar características por cada categoría e indicar 1 si la fila tiene definida la misma

alumnos	nivel	primaria	secundario	grado	posgrado
juan	prim	1	0	0	0
luis	grado	0	0	1	0
claudia	grado	0	0	1	0
mariana	secun	0	1	0	0
elena	posgrado	0	0	0	1



Tipos de variables

Cuando se genera una columna por cada categoría la técnica se denomina **one-hot encoding**

Si se evita una columna porque su valor queda definido por defecto se llaman **variables dummy**

alumnos	nivel	primaria	secundario	grado
juan	prim	1	0	0
luis	grado	0	0	1
claudia	grado	0	0	1
mariana	secun	0	1	0
elena	posgrado	0	0	0



Tipos de variables

Para variables ordinales se puede mapear valores numéricos a las categorías y evitar agregar tantas columnas

alumnos	nivel	nivel_número
juan	prim	1
luis	grado	3
claudia	grado	3
mariana	secun	2
elena	posgrado	4



Normalización de Variables

Normalizar significa, en este caso, comprimir o extender los valores de la variable para que estén en un rango definido

Algunos algoritmos de ML trabajan mejor con variables normalizadas

No hay un solo tipo de normalización que funcione siempre

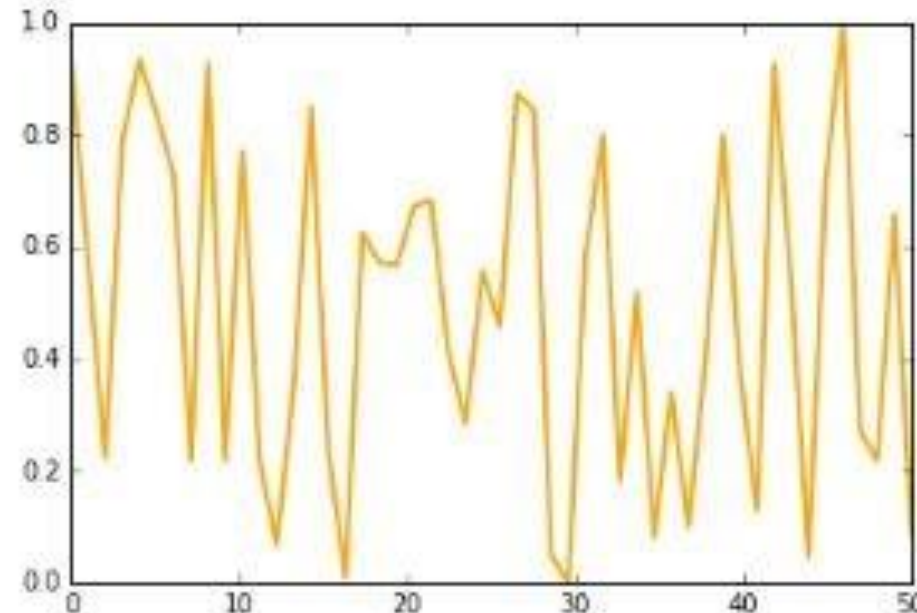
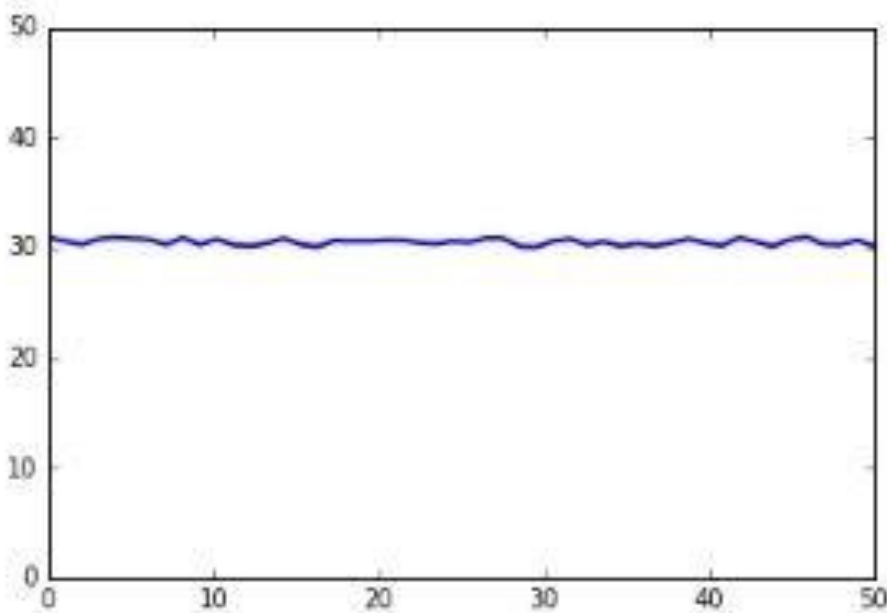
Puede fallar...

Normalización min-max

$$X_{normalized} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Problema: Comprime valores entre valores empíricos y cualquier variación será magnificada. No es buena para curvas con poca variación

Normalización de Variables



Normalización de Variables

Normalización Z-score

$$X_{normalized} = \frac{X - X_{mean}}{X_{stddev}}$$



Si la distribución es asimétrica se puede usar Z-score modificado.
Usa mediana y desviación de la mediana

Problema Normalización: Muy sensible a valores atípicos



Uso de Intervalos (bins)

Se emplea para simplificar valores continuos como discretos

Detección y Tratamiento de Datos Ausentes:

¿Qué es?

Identificar la falta de algunos de los datos en cada variable

¿Por qué es necesario?

Los datos ausentes pueden generar problemas a la hora de aplicar técnicas de machine learning, realizar análisis estadísticos o generar representaciones gráficas

Tratamiento

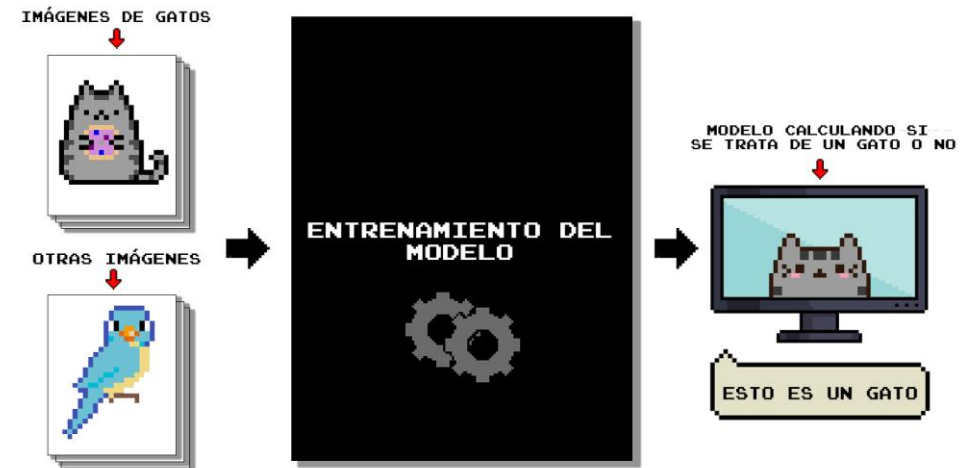
Eliminarlos Sólo si son muy pocos y la eliminación de filas no es significativa o si son muchos en una columna no muy significativa

Imputar valores sintéticos Existen varias maneras de hacerlo, como por ejemplo sustituirlos por la media o la mediana, o completar los valores faltantes con el valor anterior o posterior de la columna; o simplemente emplear un algoritmo de ML que ignore el caso (K-NN)



Detección y Tratamiento de Datos Ausentes:

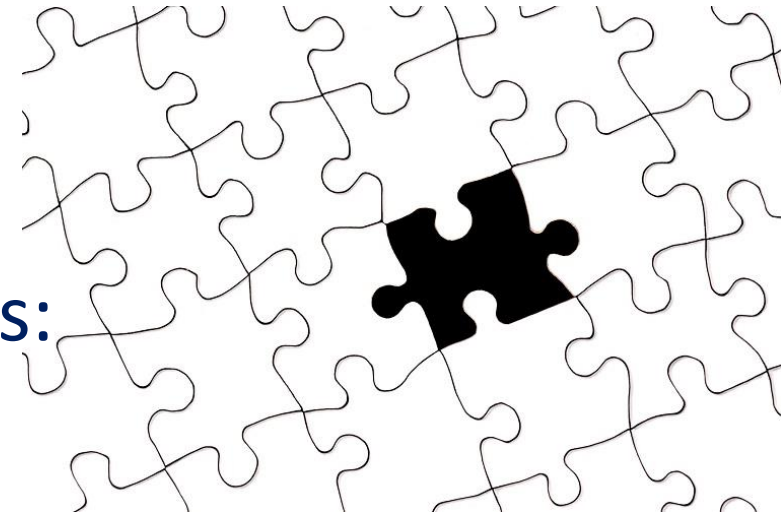
Sin embargo si los datos ausentes están en la columna/característica **objetivo** pueden salvarse perfectamente aplicando un método de ML que **no sea Supervisado** y reservar los casos/filas con valor para el conjunto de testeo (K-means, Dendogramas, Mezcla Gaussiana)



Detección y Tratamiento de Datos Ausentes:

Si se desea rellenar faltantes hay varios métodos:

- ✓ Media/mediana/moda (col)
- ✓ Hot deck (imito a los más parecidos, cercanos o familiares) (fil)
- ✓ Regresión (fil)
- ✓ Complementar con otro dataset
- ✓ Interpolación en series temporales (col)



¿Cómo imputo valores sintéticos?

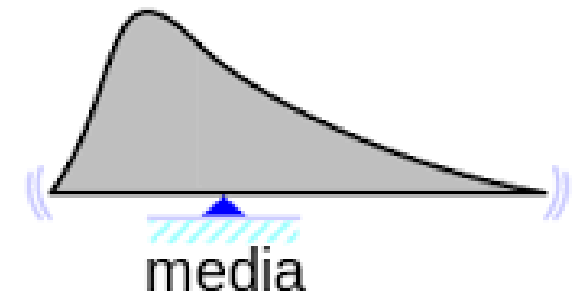
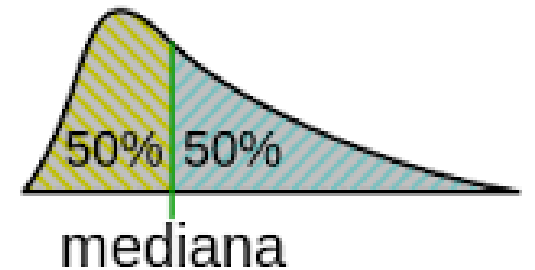
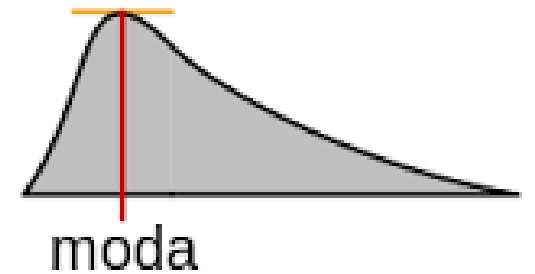
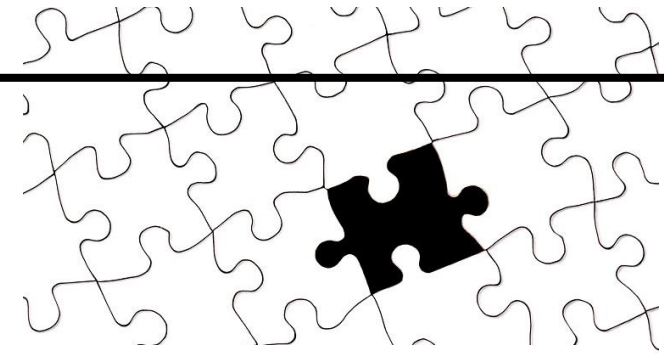
Se recomienda:

Para valores discretos imputar la **moda**
`datos [<nombreCol>].mode()`

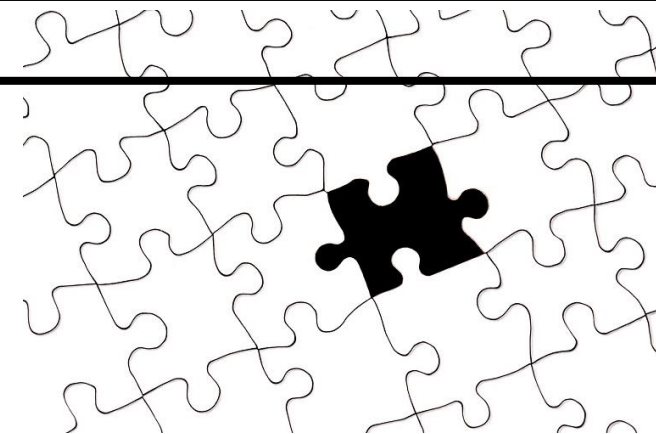
Para valores continuos imputar la **media**
`datos [<nombreCol>].mean()`

Atención!!

Siempre que agregamos campos sintéticos conviene agregar una columna marcando cuáles son originales y cuáles no



¿Cómo genero una columna con 1s y 0s marcando si el dato de la columna asociada es original o sintético?



```
nueva = <dataFrame>[<col>].isnull()
```

Me arma una lista con True/False

True si era NaN, False si tenía valor

Ahora simplemente aplico casteo entero

```
for i in range(len(nueva)) :  
    nueva[i]=int(nueva[i])
```

Ahora tengo una lista con 1s en las posiciones correspondientes a las filas en que tenía NaN y 0s en el resto

Solo falta agregar esta columna a mi dataframe

¿Cómo genero una columna con 1s y 0s marcando si el dato de la columna asociada es original o sintético?

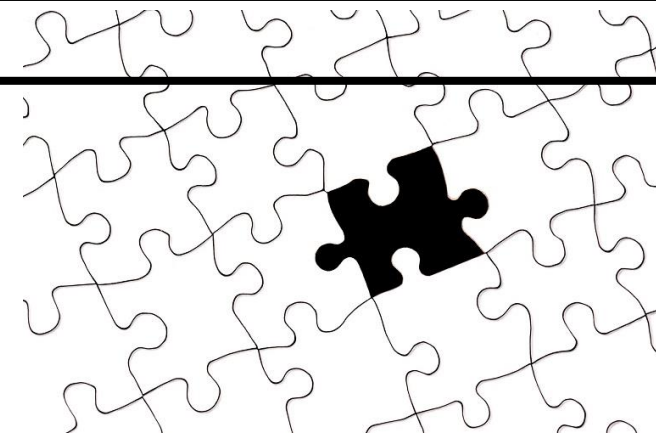
Más detallado (otra de las mil formas de hacerlo...) Vamos a generar 1s y 0s según la persona del dataset sea mayor o menor de edad

```
import pandas as pd
personas=pd.DataFrame([['Juan',15,10],
                        ['María',27,8.40],
                        ['Rosario',25,8],
                        ['Pedro',21,9.80],
                        ['Chiara',14,9.40]],
                      columns=['Nombre','Edad','Promedio'])
print(personas)
```

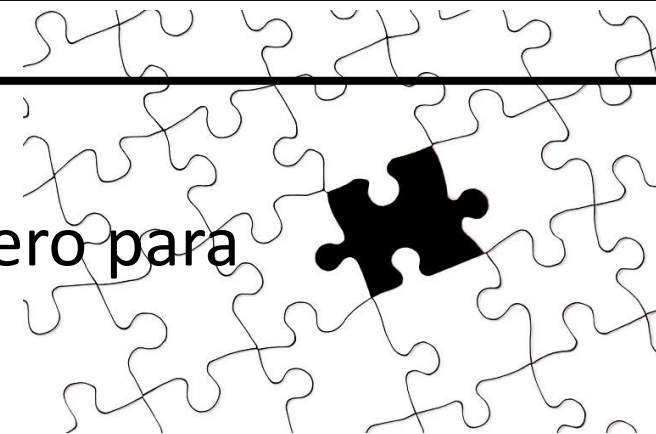


Así queda mi DataFrame personas

	Nombre	Edad	Promedio
0	Juan	15	10.0
1	María	27	8.4
2	Rosario	25	8.0
3	Pedro	21	9.8
4	Chiara	14	9.4



Ahora voy a agregar una columna en la posición que quiero para que me quede a la derecha de la que señalo

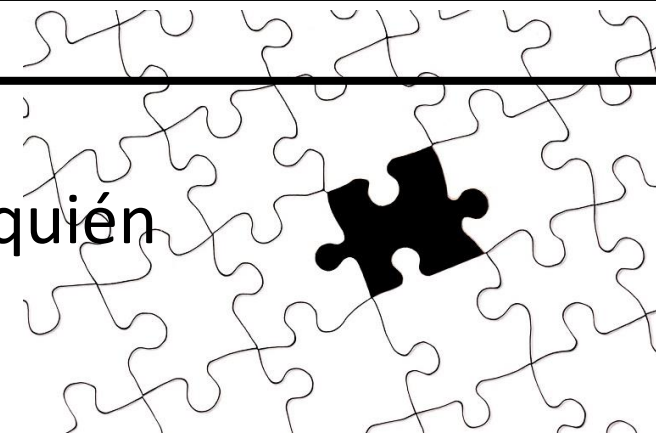


```
personas.insert(2, 'Mayoria', True)
```

	Nombre	Edad	Mayoria	Promedio
0	Juan	15	True	10.0
1	María	27	True	8.4
2	Rosario	25	True	8.0
3	Pedro	21	True	9.8
4	Chiara	14	True	9.4



Usamos la nueva columna para marcar, en base a la edad quién
Es mayor y quién no



```
personas= personas.assign(Mayoria = personas['Edad'] >20)
```

	Nombre	Edad	Mayoria	Promedio
0	Juan	15	False	10.0
1	María	27	True	8.4
2	Rosario	25	True	8.0
3	Pedro	21	True	9.8
4	Chiara	14	False	9.4



Finalmente si quiero que la columna Mayoria quede numérica, con 1 si es mayor de edad y 0 si no lo es, debemos convertir el tipo de la columna

```
personas['Mayoria'] = personas['Mayoria'].astype(int)
```

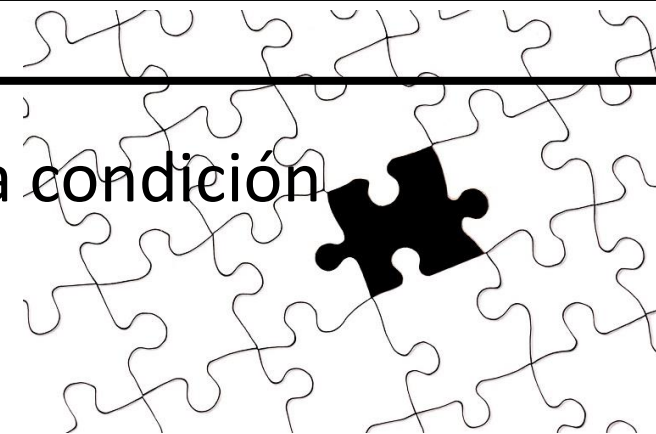
	Nombre	Edad	Mayoria	Promedio
0	Juan	15	0	10.0
1	María	27	1	8.4
2	Rosario	25	1	8.0
3	Pedro	21	1	9.8
4	Chiara	14	0	9.4

Para no desesperar... El ejemplo anterior generando una columna que marque cuál será sintético (originalmente faltante, con 1) y cuál no (con 0) Cambiamos un poco el DataFrame de ejemplo para introducir faltantes en una columna

	Nombre	Edad	Promedio
0	Juan	15	10.0
1	María	27	8.4
2	Rosario	25	NaN
3	Pedro	21	9.8
4	Chiara	14	NaN

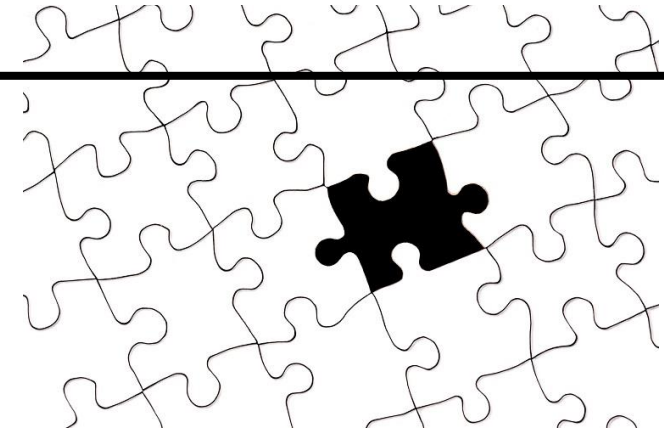


Aplicamos una secuencia parecida de sentencias, ahora la condición a detectar es faltante en la columna Promedio



```
personas.insert(3, 'Sintetico', True)
personas=personas.assign
    (Sintetico = personas['Promedio'].isnull())
personas['Sintetico'] = personas['Sintetico'].astype(int)
```

	Nombre	Edad	Promedio	Sintetico
0	Juan	15	10.0	0
1	María	27	8.4	0
2	Rosario	25	NaN	1
3	Pedro	21	9.8	0
4	Chiara	14	NaN	1



¿Siempre va moda o media?

Partamos del siguiente Dataframe **nuevaCli**:

	apellido	edad	ciudad	provincia
0	Antón	50.0	Chajarí	Entre Ríos
1	Khunter	72.0	Concordia	Entre Ríos
2	Ortúzar	44.0	La Carlota	Córdoba
3	Gómez	23.0	Chajarí	Entre Ríos
4	Arce	35.0	La Paz	Entre Ríos
5	Alves	NaN	Salsipuedes	Córdoba
6	Alves	41.0	None	None
7	Díaz	72.0	Tafí Viejo	Tucumán

48.142857

```
nuevaCli['edad'].fillna(int(nuevaCli['edad'].mean()),  
                           inplace=True)
```

¿Siempre va moda o media?

Si los faltantes son muchos quizás convenga rellenarlos con valores aleatorios calculados con una distribución normal de media y desviación estándar igual a la de los datos que si están, en lugar de la media

Volvamos al Dataframe **nuevaCli** de ejemplo

	apellido	edad	ciudad	provincia
0	Antón	50.000000	Chajarí	Entre Ríos
1	Khunter	53.265342	Concordia	Entre Ríos
2	Ortúzar	44.000000	La Carlota	Córdoba
3	Gómez	23.000000	Chajarí	Entre Ríos
4	Arce	35.000000	La Paz	Entre Ríos
5	Alves	53.265342	Salsipuedes	Córdoba
6	Alves	53.265342	None	None
7	Díaz	72.000000	Tafí Viejo	Tucumán

¿Cómo imputo valores aleatorios?

```
import random as rd
mu=nuevaCli['edad'].mean()
sigma= nuevaCli['edad'].std()
nuevaCli['edad'].fillna(
    rd.gauss (mu,sigma),inplace=True)
```

53.265342

Si la vamos a hacer, hagámosla bien

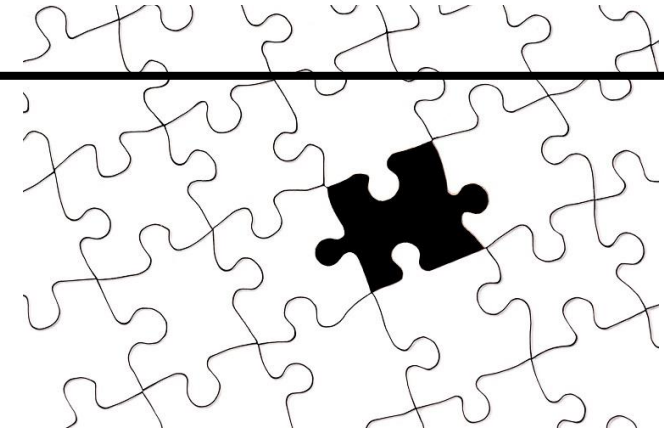
Necesito un bucle que recorra las filas con celdas faltantes y tire un aleatorio diferente para cada una

```
filasFaltante= nuevaCli[nuevaCli['edad'].isnull()]\nfor i, fila in filasFaltante.iterrows():\n    nuevaCli.loc[i, 'edad']=int(rd.gauss(mu, sigma))
```

	apellido	edad	ciudad	provincia
0	Antón	50.0	Chajarí	Entre Ríos
1	Khunter	39.0	Concordia	Entre Ríos
2	Ortúzar	44.0	La Carlota	Córdoba
3	Gómez	23.0	Chajarí	Entre Ríos
4	Arce	35.0	La Paz	Entre Ríos
5	Alves	60.0	Salsipuedes	Córdoba
6	Alves	21.0	None	None
7	Díaz	72.0	Tafí Viejo	Tucumán

**Coloco aleatorios pero
respetando distribución**





¿Siempre va moda o media?

En caso de una columna entera:

	apellido	edad	ciudad	provincia
0	Antón	50.0	Chajarí	Entre Ríos
1	Khunter	NaN	Concordia	Entre Ríos
2	Ortúzar	44.0	La Carlota	Córdoba
3	Gómez	23.0	Chajarí	Entre Ríos
4	Arce	35.0	La Paz	Entre Ríos
5	Alves	63.0	Salsipuedes	Córdoba
6	Alves	21.0	None	None
7	Díaz	21.0	Tafí Viejo	Tucumán

Moda 21.0

```
moda=nuevaCli['edad'].mode().values[0]  
nuevaCli['edad'].fillna(modas,inplace=True)
```

¿Siempre va moda o media?

Pero también se puede generar aleatorios según la distribución de frecuencias...



	apellido	edad	ciudad	provincia
0	Antón	50.0	Chajarí	Entre Ríos
1	Khunter	NaN	Concordia	Entre Ríos
2	Ortúzar	44.0	La Carlota	Córdoba
3	Gómez	21.0	Chajarí	Entre Ríos
4	Arce	21.0	La Paz	Entre Ríos
5	Alves	NaN	Salsipuedes	Córdoba
6	Alves	NaN	None	None
7	Díaz	72.0	Tafí Viejo	Tucumán

```
import random as rd
'''obtengo un array con los valores no
nulos de la columna edad'''
elegir=nuevaCli['edad'].dropna().values
nuevaCli['edad'].fillna(
    rd.choice(elegir),inplace=True)
```





¿Siempre va moda o media?

Pero también se puede generar aleatorios según la distribución de frecuencias...

	apellido	edad	ciudad	provincia
0	Antón	50.0	Chajarí	Entre Ríos
1	Khunter	21.0	Concordia	Entre Ríos
2	Ortúzar	44.0	La Carlota	Córdoba
3	Gómez	21.0	Chajarí	Entre Ríos
4	Arce	21.0	La Paz	Entre Ríos
5	Alves	21.0	Salsipuedes	Córdoba
6	Alves	21.0	None	None
7	Díaz	72.0	Tafí Viejo	Tucumán

```
'''Una vez que no hay NaN puedo  
convertir la columna edad a int'''  
nuevaCli['edad']=  
nuevaCli['edad'].astype(int)
```

Ojo!!!

Vale lo mismo que con la media

**Si quiero valores
diferentes empleo bucle**

Hot Deck

Para aplicar hot-deck a valores faltantes, primero se divide la muestra en grupos (**mazos**) basados en características similares. Luego, se selecciona un valor de un miembro del mazo para imputar los valores faltantes. Este proceso puede ser aleatorio dentro de cada mazo, basado en criterios de similitud o moda, media, como siempre

Pasos detallados:

- 1. Identificar y categorizar los datos faltantes:** Determinar qué datos faltan y en qué variables
- 2. Definir los criterios de agrupación (mazos):** Elegir variables que ayuden a crear grupos homogéneos. Por ejemplo, se puede agrupar por edad, género, ubicación, etc.
- 3. Seleccionar un donante (registro con valores completos):** Para cada valor faltante, se busca un donante dentro del mismo mazo. El donante puede ser seleccionado aleatoriamente, utilizando métricas de distancia para encontrar el más similar o emplear medidas descriptivas centrales
- 4. Imputar el valor faltante:** El valor del donante seleccionado se usa para reemplazar el valor faltante
- 5. Repetir el proceso:** Se repiten los pasos anteriores para todos los valores faltantes



La principal desventaja de este método es que todo el éxito radica en la correcta división en grupos (mazos)

Tengo que entender muy bien la dinámica de los datos y no pifiarle al determinar cuáles son las características cruciales que definen grupos de pertenencia

Hot Deck

La buena noticia (si no sé cómo organizar los grupos) es que hay un algoritmo de ML que lo puede hacer por mí (K-means)

Ejemplo inapropiado:

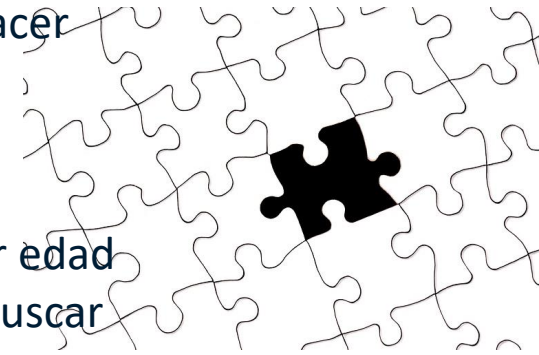
Supongamos que tienes una variable **Salario** con valores faltantes. Puedes agrupar a las personas por edad y género (mazos). Si tienes un valor faltante de salario para una persona de 30 años, mujer, puedes buscar en el grupo de mujeres de 30 años, seleccionar aleatoriamente un salario de esa categoría y usarlo para imputar el valor faltante

Probablemente esa decisión produzca lo que se conoce como **sesgo**

¿Fue adecuada esa división en grupos? ¿realmente sexo y edad condicionan sueldo?

Ejemplo apropiado:

Supongamos que tienes una variable **peso** con valores faltantes. Puedes agrupar a las personas por edad, altura y género (mazos). Si tienes un valor faltante de peso para una persona de 30 años, mujer, y de 1,65 m puedes buscar en el grupo de mujeres de 30 años y altura media, seleccionar la media de peso de ese grupo y usarla para imputar el valor faltante



Regresión

Relación entre variables:

La regresión busca modelar la relación entre una variable dependiente (la que queremos predecir) y una o más variables independientes (las que pueden influir en la variable dependiente)

Variables independientes:

Estas variables pueden ser numéricas, categóricas o una combinación de ambas. **Pero las categóricas serán convertidas a una representación numérica**

Variable dependiente:

Esta variable siempre es numérica y representa el valor que queremos predecir

Tipos de regresión:

Existen diferentes tipos de regresión, como la lineal, polinómica, exponencial, etc., cada uno con una forma diferente de modelar la relación entre las variables

Función de regresión:

La función de regresión se obtiene ajustando las observaciones a una función elegida, generalmente utilizando el método de Mínimos Cuadrados (MCO)

En este método, a diferencia de la mayoría, se ajusta el dato en función de otros datos de la misma fila

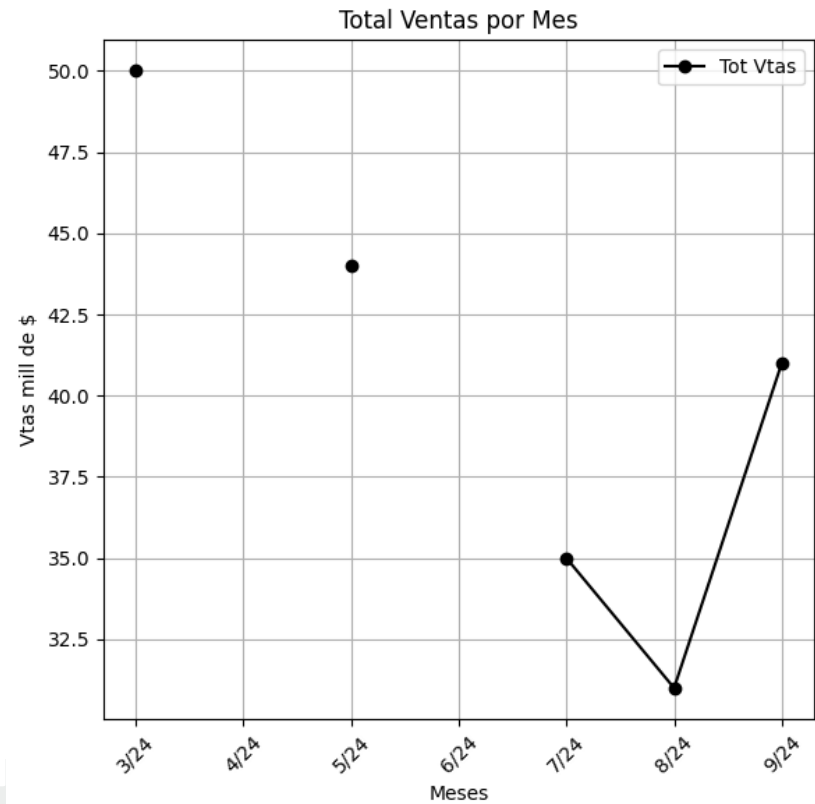


Interpolación

La Interpolación se usa sobre todo en series temporales. Donde tiene más relevancia los valores cercanos que el promedio general

Ejemplo (Dataframe **ventas**):

	Mes	Total Vendido	Status Envío
0	3/24	50.0	Despachado
1	4/24	NaN	Despachado
2	5/24	44.0	Despachado
3	6/24	NaN	No Despachado
4	7/24	35.0	No Despachado
5	8/24	31.0	No Despachado
6	9/24	41.0	Despachado



```
ventas=ventas.interpolate()
```

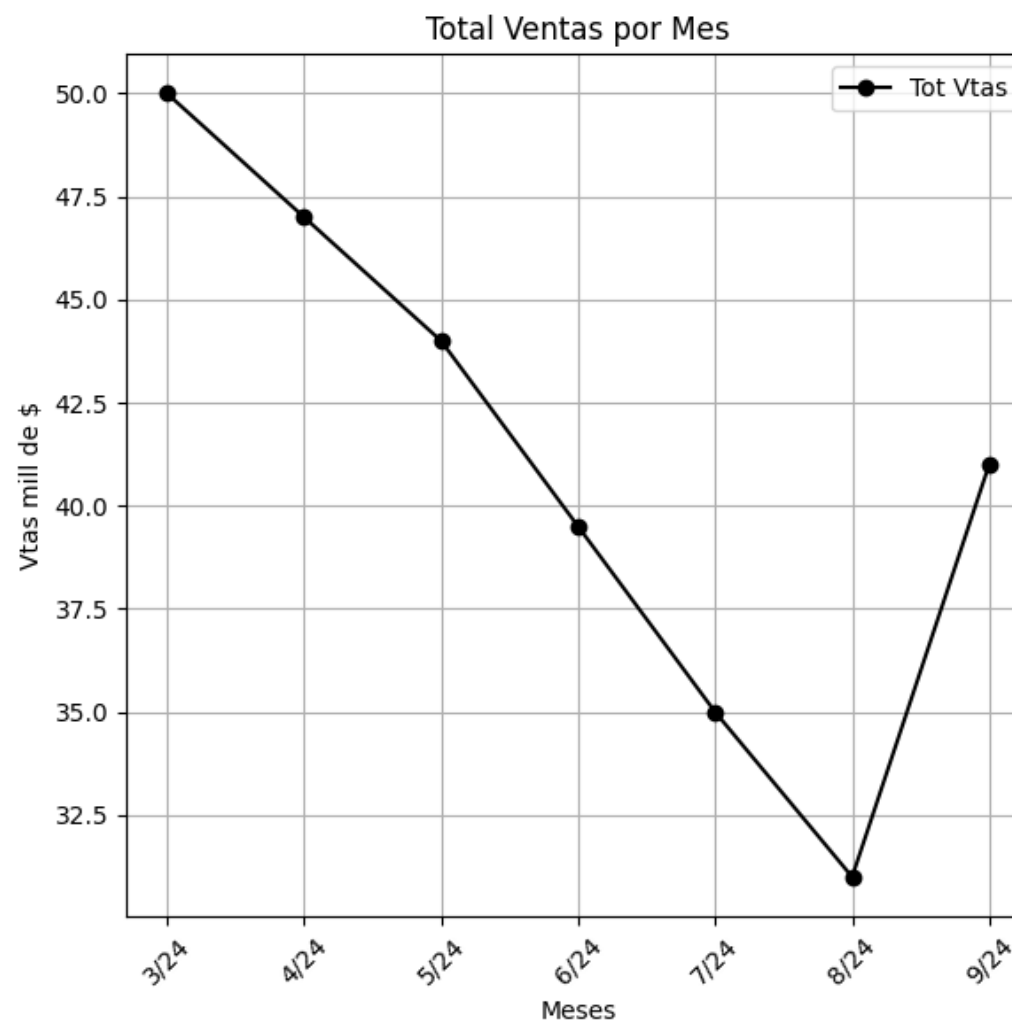


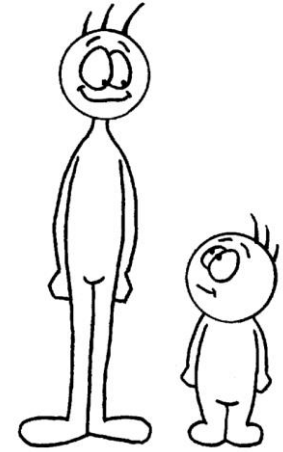
Interpolación

La Interpolación se usa sobre todo en series temporales. Donde tiene más relevancia los valores cercanos que el promedio general

Ejemplo (Dataframe **ventas**):

	Mes	Total Vendido	Status Envío
0	3/24	50.0	Despachado
1	4/24	47.0	Despachado
2	5/24	44.0	Despachado
3	6/24	39.5	No Despachado
4	7/24	35.0	No Despachado
5	8/24	31.0	No Despachado
6	9/24	41.0	Despachado





Identificación de Datos Atípicos:

¿Qué es?

Identificar datos con valores significativamente distintos a los que presenta la variable en general

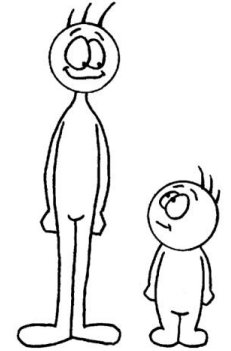
Los llamamos **outliers**, por contraposición a los datos típicos o **inliers**

¿Por qué es necesario su tratamiento?

Pueden modificar los resultados y restar potencia a los análisis estadísticos o técnicas de machine learning aplicadas

Tratamiento

Disminuir su influencia en análisis posteriores o, en casos muy extremos, eliminarlos del conjunto de datos



Identificación de Datos Atípicos:

Globales: Son extremos para todo el conjunto

Ej: altura persona 3 m

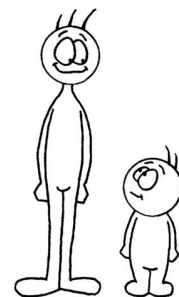
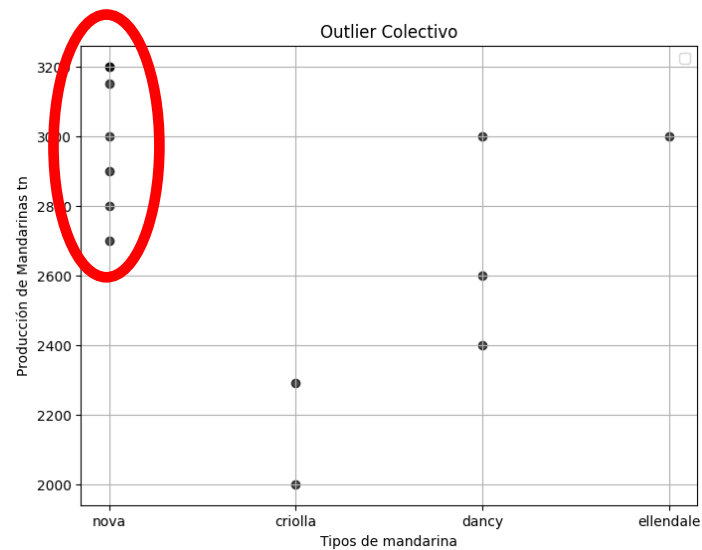
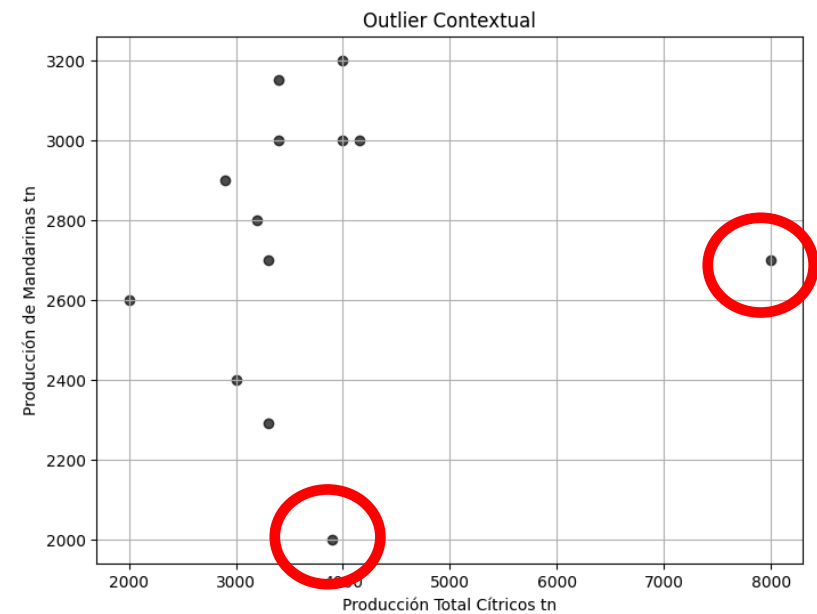
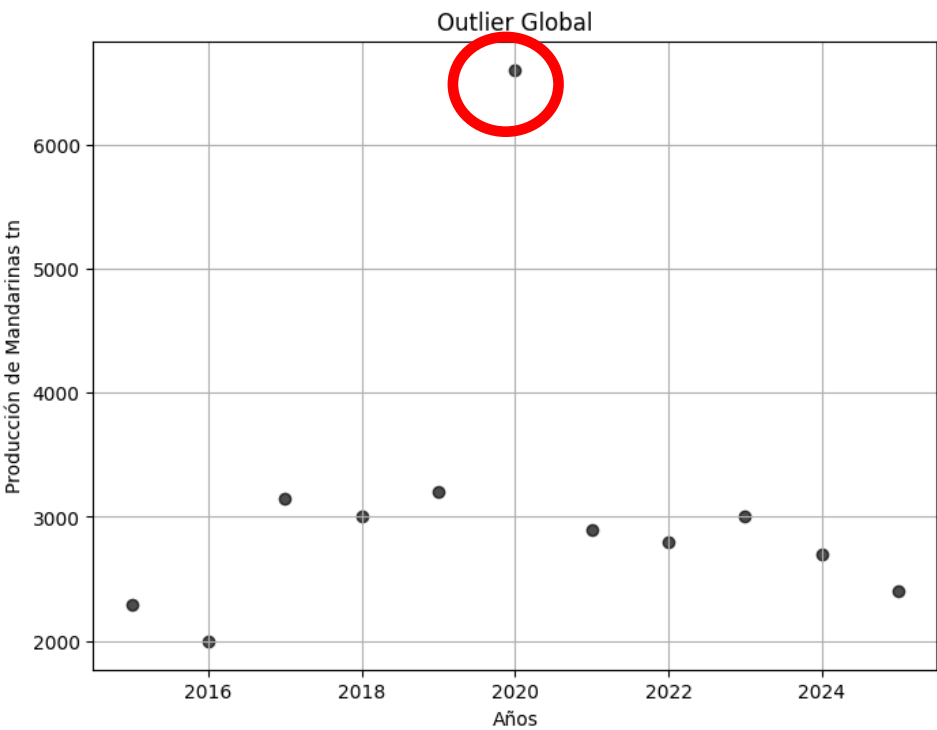
Contextuales: No son atípicos en el conjunto pero si para un contexto determinado

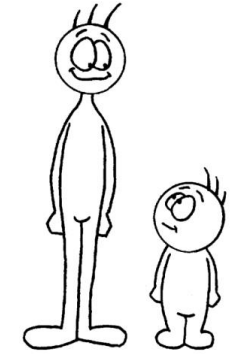
Ej: peso persona 55 kg, pero para el grupo de los extraordinariamente altos (altura de más de 2 m)

Colectivos: No son atípicos si el desvío es individual, pero todos???

Ej: En un determinado momento en un restaurante la mitad de los comensales son veganos

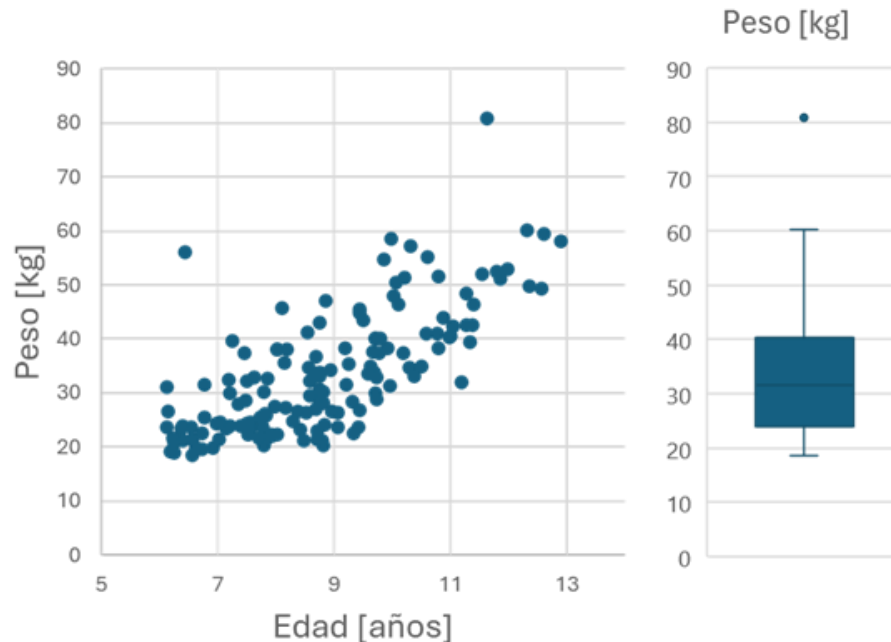
Identificación de Datos Atípicos:



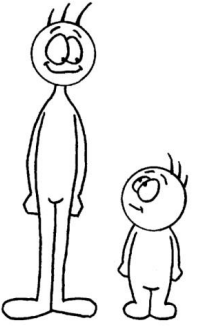


Identificación de Datos Atípicos:

¿Cuán alejado de la media debe estar el valor para ser considerado outlier?
Habitualmente, en poblaciones normales se considera que 1,5 veces la desviación estándar de distancia es moderadamente atípico, 3 es atípico



Los outliers contextuales no se observan en gráficos univariados



Identificación de Datos Atípicos:

Los outliers contextuales deben ser buscados en un análisis multivariado

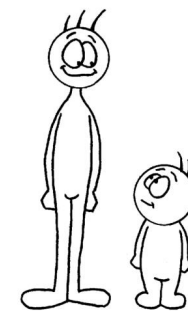
Pero, claro, cómo se mide la distancia hacia la media para saber si califica como valor atípico?

Con la distancia euclídeana pueden realizarse apreciaciones incorrectas

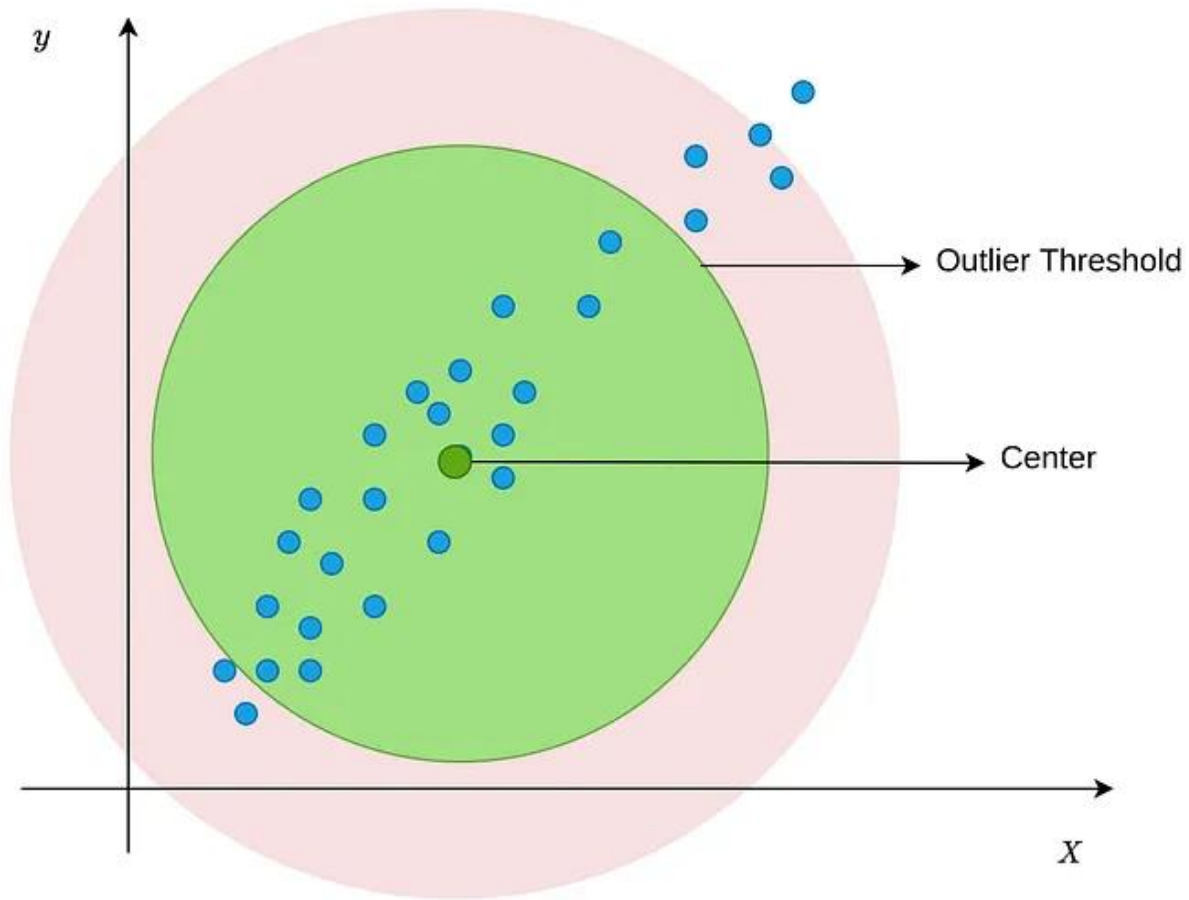
En su lugar la distancia de Mahalanobis es más adecuada, sobre todo si las variables están correlacionadas, ya que esta medida considera y acompaña el desplazamiento correlacionado de las variables

La distancia de Mahalanobis es una medida estadística utilizada para cuantificar la separación entre dos puntos, teniendo en cuenta no sólo la separación entre estos, sino también la covarianza entre las variables del conjunto de datos. A diferencia de la distancia euclídea, en la que se asume independencia entre las variables

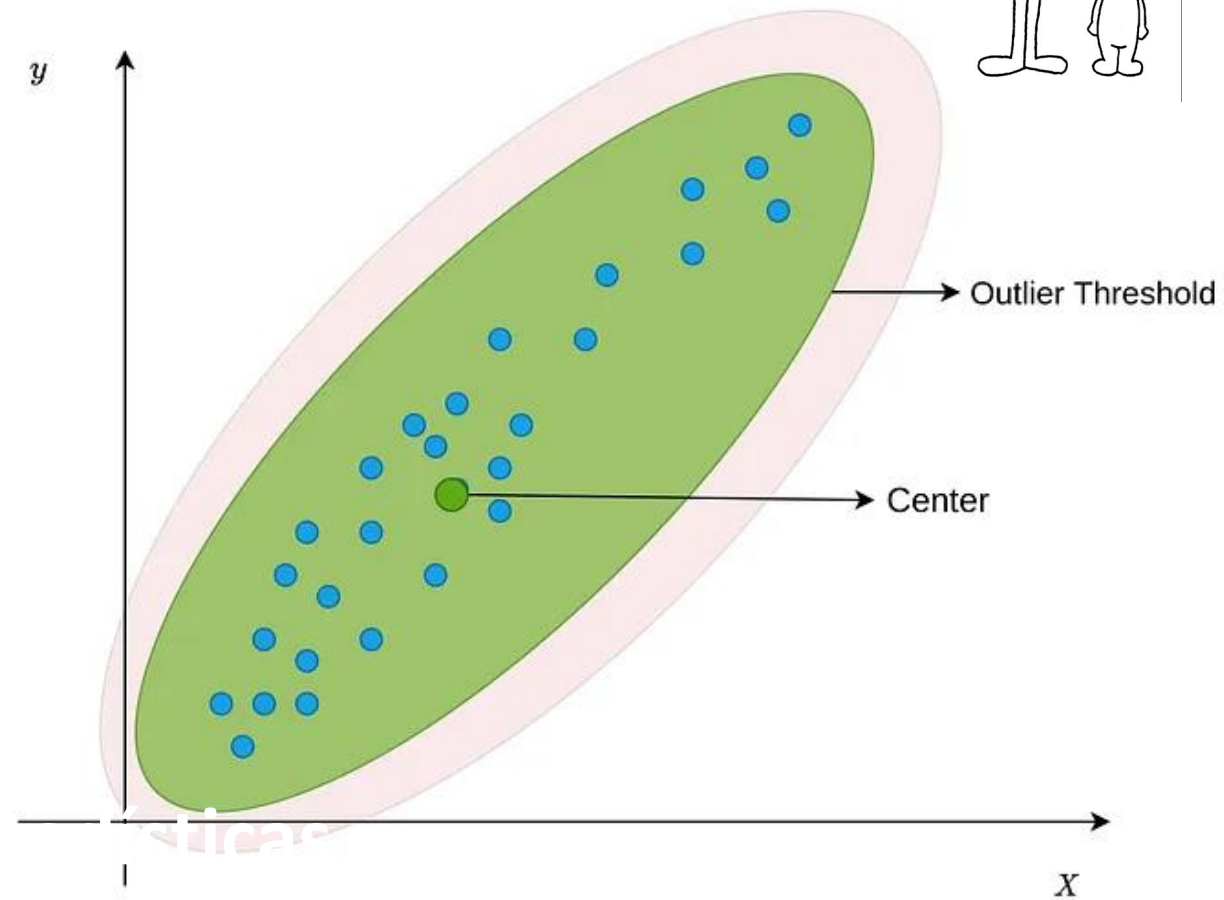
Cuando no existe correlación ambas distancias tienden a coincidir

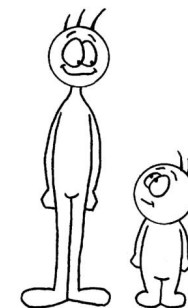


Euclidean



Mahalanobis

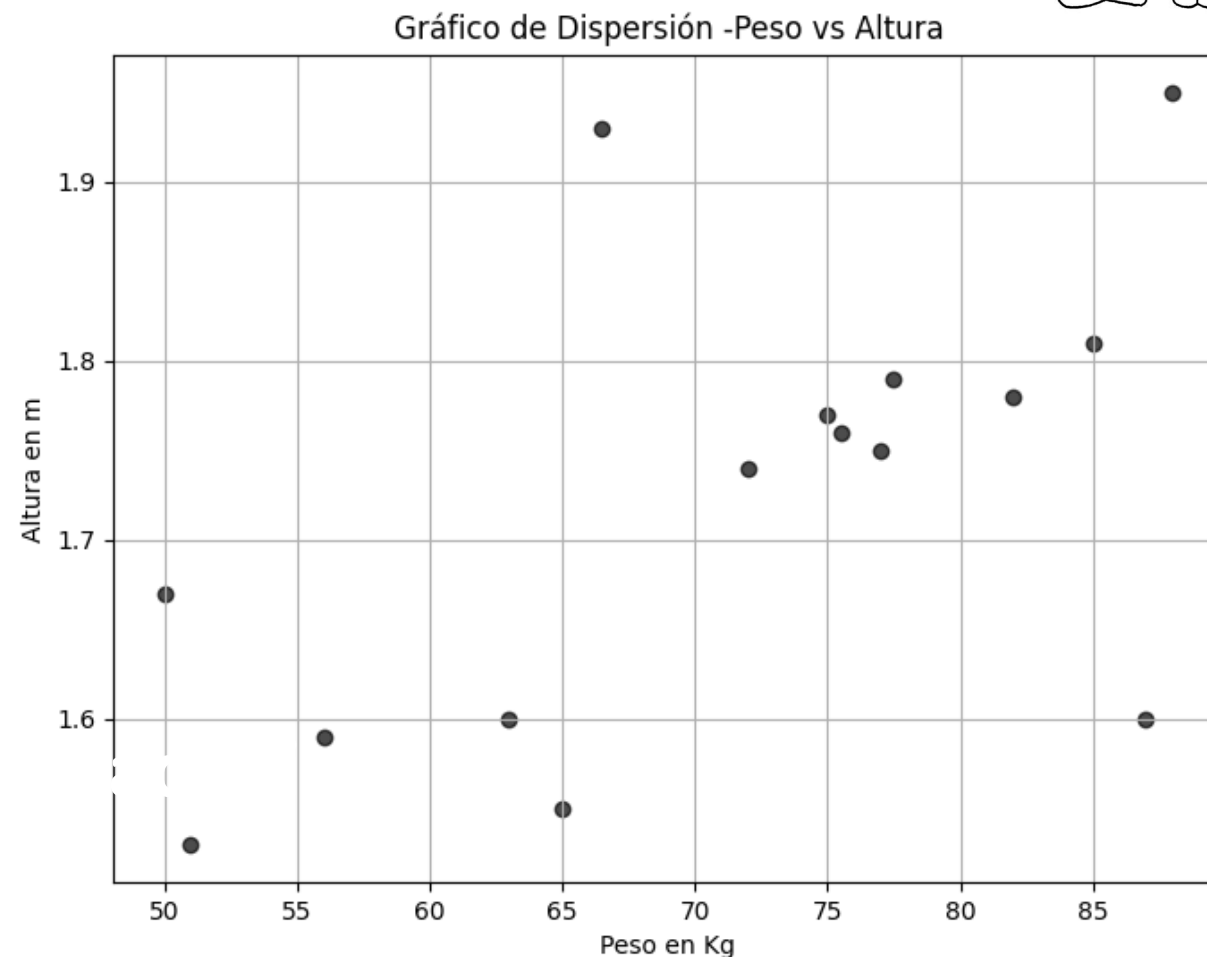




¿Cómo calculo Mahalanobis?

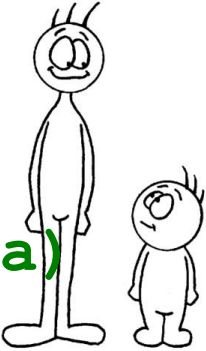
Dataframe **datosBio**:

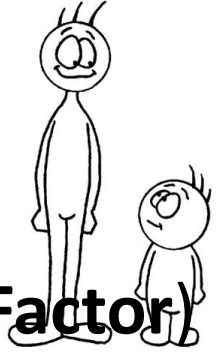
	Nombre	Peso	Altura
0	juan	66.5	1.93
1	Ana	63.0	1.60
2	Luis	88.0	1.95
.....			
11	Marga	87.0	1.60
12	Ileana	51.0	1.53
13	Joaco	77.0	1.75
14	Andy	75.0	1.77



```
import numpy as np
from scipy.spatial import distance
# Seleccionamos solo las variables numéricas (Peso y Altura)
X = datosBio[['Peso', 'Altura']]
media = X.mean().values # Media de los datos
cov=X.cov() # Matriz de covarianza
# Inversa de la matriz de covarianza
invCov = np.linalg.inv(cov)
x = X.iloc[0].values # Punto específico
# Distancia de Mahalanobis
mahalDist = distance.mahalanobis(x, media, invCov)
print(f"Distancia de Mahalanobis del punto {x} a la
media: {mahalDist:.4f}")
```

Distancia de Mahalanobis del punto [66.5 1.93] a la media: 2.2780





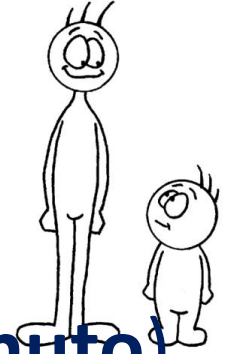
Identificación de Datos Atípicos:

Otra técnica interesante para capturar outliers es **LOF (Local Outlier Factor)**

Compara la densidad del vecindario de cada punto (cuántos puntos tiene cerca) con la de sus vecinos (considerando vecinos a otros casos parecidos o cercanos en los valores de las características comparadas)

Se basa en el algoritmo de k-vecinos más cercanos (k-NN) de ML. Para aplicarla se necesita definir el tamaño de k, definir con qué función de distancia se trabajará y especificar el umbral de casos raros estimados (cuántos outliers supongo que hay en total).

LOF arroja un factor de densidad. Una densidad alta (4) significa que los demás tienen 4 veces más vecinos que ese caso. Por lo tanto parece un **caso aislado** o **outlier**.



Un ejemplo para entender mejor cómo funciona:

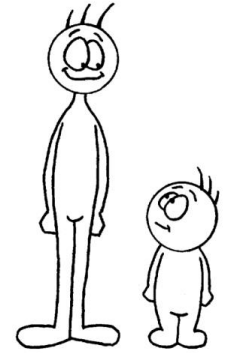
Analizaremos el indicador **BPM (Latidos del corazón por minuto)**

en pacientes de edades cercanas a los 20 años

Lo normal para esa edad, estando en reposo, es tener entre 60 y 80 latidos por minuto

Si n personas tienen entre 65 y 75 latidos, están **cerca** (se parecen) porque sus números son casi iguales. Forman un grupo denso

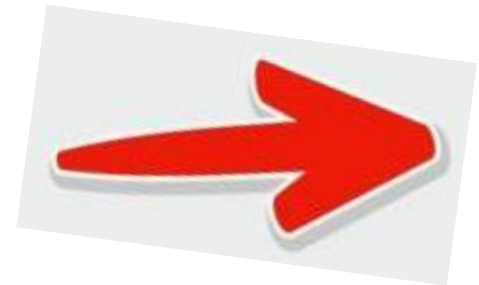
Si una persona de 20 años tiene 140 latidos, su número está **lejos** de los demás. Es un **outlier**

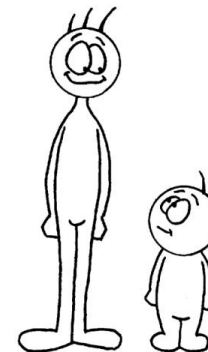


Un ejemplo para entender mejor cómo funciona:

```
import numpy as np
import pandas as pd
from sklearn.neighbors import LocalOutlierFactor

# Datos: Solo el número de latidos (BPM)
# Primeros 5 pacientes son saludables/normales. El último es la anomalía.
bpm = np.array([[70], [72], [68], [71], [69], [140]])
modelo = LocalOutlierFactor(n_neighbors=2, contamination=0.15)
predicciones = modelo.fit_predict(bpm)
# Convertimos el puntaje interno a positivo para leerlo fácil
puntajesLOF = -modelo.negative_outlier_factor_
# Creamos la tabla de resultados
tabla = pd.DataFrame(bpm, columns=['Latidos BPM'])
tabla['Prediccion'] = predicciones
tabla['Puntaje_LOF'] = puntajesLOF
```





Un ejemplo para entender mejor cómo funciona:

	Latidos BPM	Prediccion	Puntaje_LOF
0	70	1	0.666667
1	72	1	1.250000
2	68	1	1.250000
3	71	1	1.250000
4	69	1	1.250000
5	140	-1	45.666667



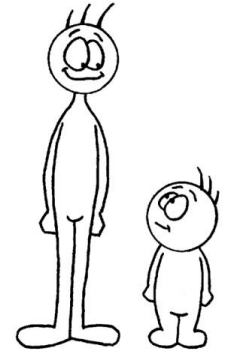
1 inlier

-1 outlier

¿Cómo interpreto un LOF de 45?
Tiene 45 veces menos vecinos que el resto o tengo que estirarme 45 veces en la distancia para encontrar k vecinos

El ejemplo es brutal un LOF superior a 3 ya se considera outlier





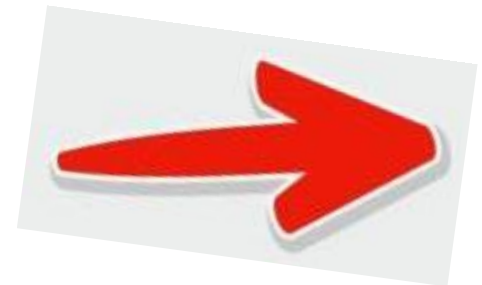
¿Cuál k es correcto?

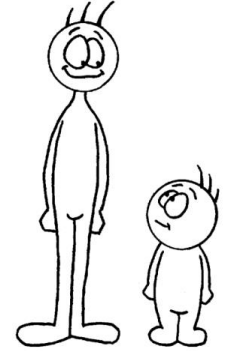
```
bmp = np.array([[70], [72], [68], [71], [69], [150], [148]])
```

```
# k muy bajo (Efecto espejo o engaño)
modBajo = LocalOutlierFactor(n_neighbors=1, contamination=0.25)
predBajo = modBajo.fit_predict(bmp)
```

```
#k adecuado (Mira el panorama completo)
modBueno = LocalOutlierFactor(n_neighbors=4, contamination=0.25)
predBueno = modBueno.fit_predict(bmp)
```

```
# Juntamos todo en una tabla para comparar
tabla = pd.DataFrame(bmp, columns=['Latidos BPM'])
tabla['Con 1 Vecino'] = predBajo
tabla['Con 4 Vecinos'] = predBueno
```





¿Cuál k es correcto?

	Latidos BPM	Con 1 Vecino	Con 4 Vecinos
0	70	1	1
1	72	1	1
2	68	1	1
3	71	1	1
4	69	1	1
5	150	1	-1
6	148	1	-1

¿Cuál es el k correcto?

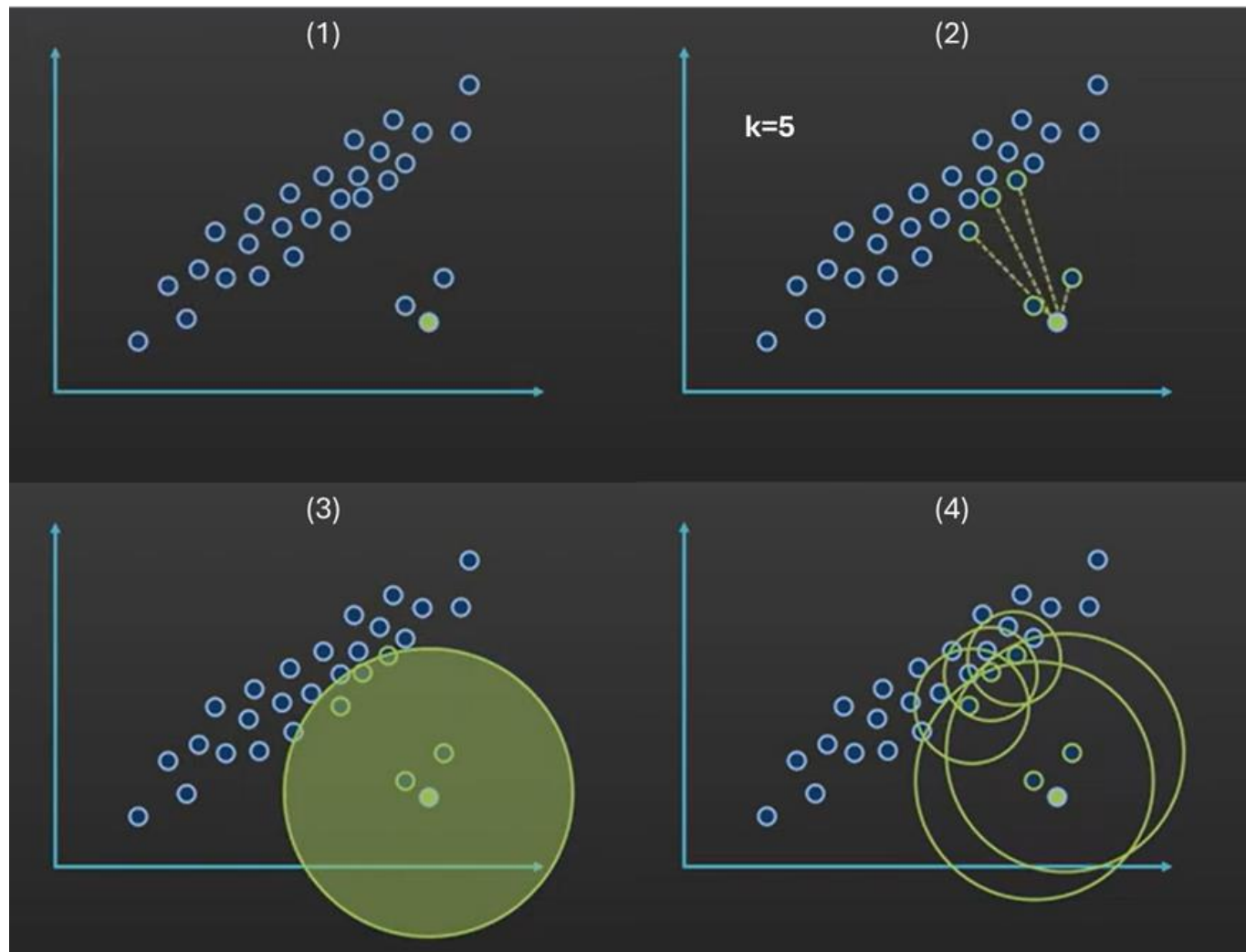
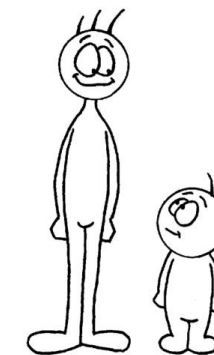
K bajo no detecta bien outliers, cualquier reunión pequeña de díscolos es tomada como casos típicos

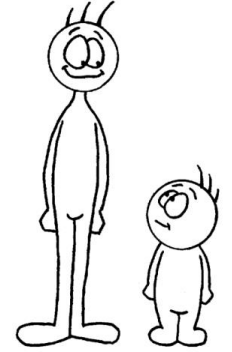
K alto requiere demasiado cálculo y puede llevar a sobreajustes (qué tal si en la tabla están los BPM de algunos bebés?)

Si no tengo muy claro k=20



Identificación de Datos Atípicos:





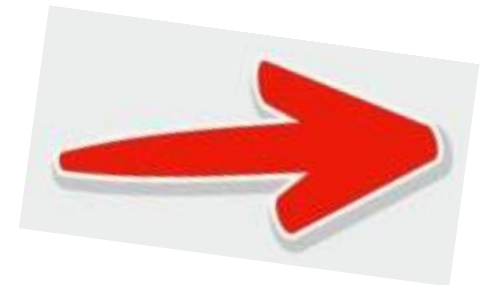
¿En cuánto debo ajustar **contamination**?

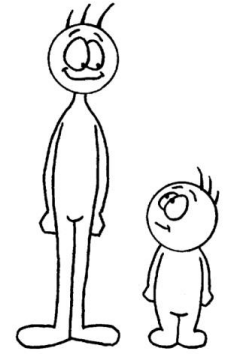
Si conoces el número exacto de outliers en tus datos reales se coloca el porcentaje en formato tasa

Si no lo conoces, el parámetro **contamination** se convierte en un desafío porque es una estimación a ciegas

Si pones un valor muy alto, marcas datos buenos como malos (falsos positivos)

Si es muy bajo, se te escapan las anomalías (falsos negativos)





¿Cómo interpretarlo?

Un médico analizando el nivel de estrés de un grupo de personas:

k es el grupo de control

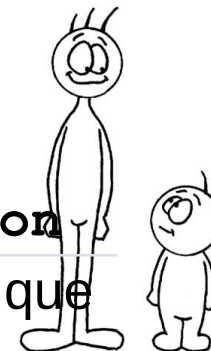
Determina con cuántas personas del entorno vas a comparar a cada paciente para ver si sus niveles son normales o raros

Si $k=5$, miras a sus 5 vecinos numéricos más cercanos

contamination es la capacidad de camas del hospital

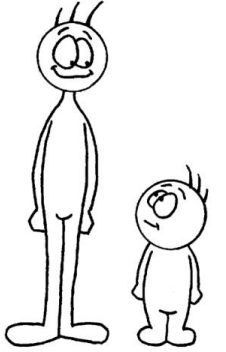
Si configuras $\text{contamination}=0.05$, le estás diciendo al sistema que no importa qué tan estresados estén todos, solo tengo espacio para diagnosticar e internar al 5% de los pacientes más graves





Característica	Parámetro k ($n_neighbors$)	Parámetro $contamination$
¿Qué controla?	El tamaño del vecindario local	El porcentaje total de alertas que vas a recibir
¿Cuándo actúa?	Al principio: Durante el cálculo matemático de la densidad de los datos	Al final: Para fijar el umbral de corte una vez que ya se conocen todas las puntuaciones
Si lo aumentas...	El modelo ignora micro-grupos de anomalías y mira el panorama más global	El modelo se vuelve más estricto y marcará más datos como outliers (baja el umbral de exigencia)
Si lo disminuyes...	El modelo se vuelve muy sensible al ruido local y a datos individuales aislados	El modelo se vuelve más permisivo y solo marcará los outliers más extremos (sube el umbral)





Reglas prácticas para fijar **contamination**

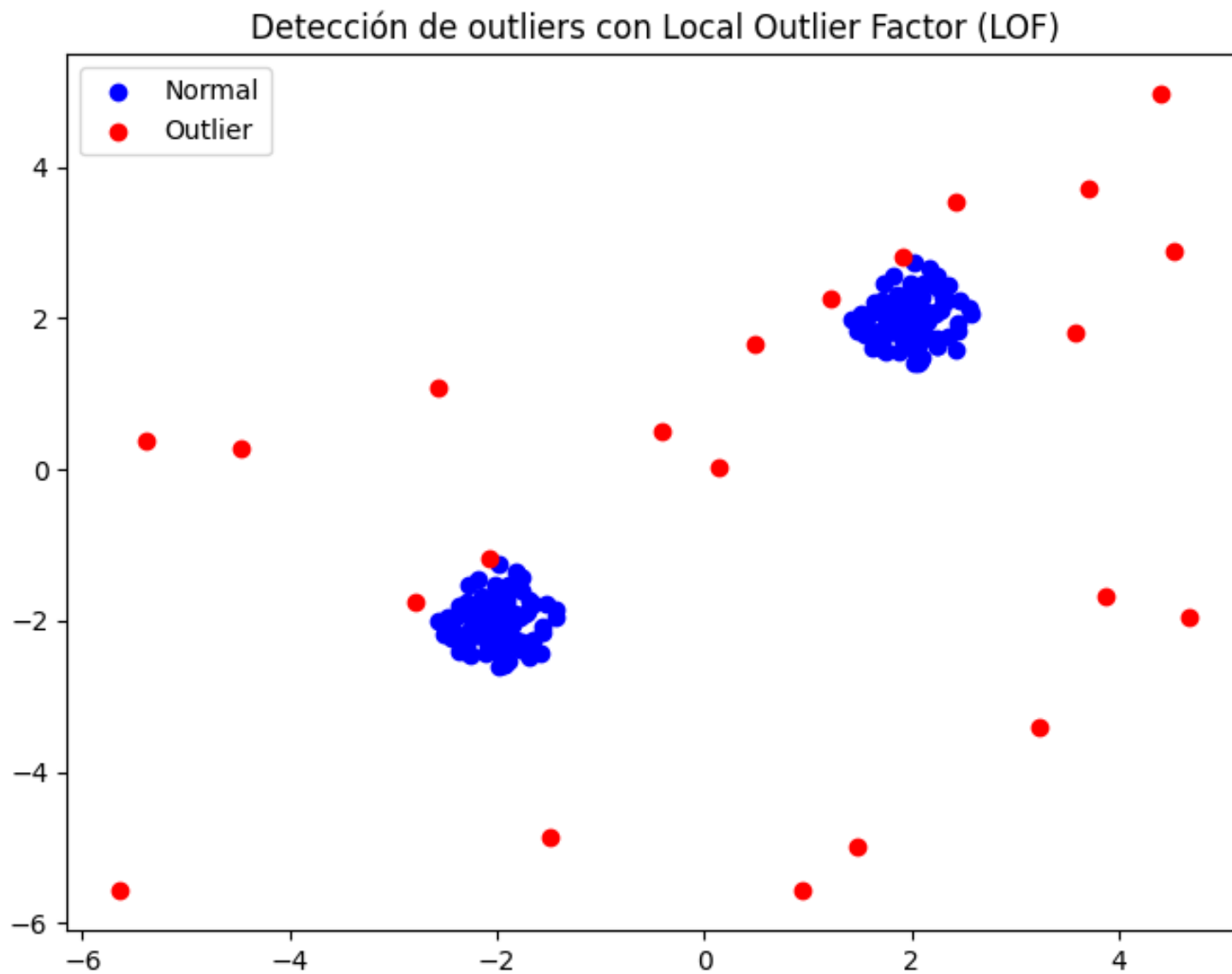
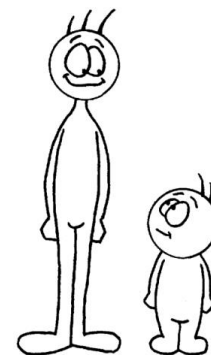
Estrategia 1: Enfoque visual

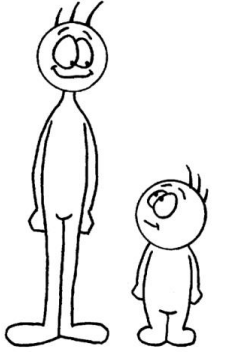
Si tus datos tienen 2 o 3 columnas, la mejor forma es probar diferentes valores de **contamination** y graficar los resultados marcando outliers con otro color. A ojómetro te das cuenta cuál valor va

Estrategia 2: Método del Codo (Elbow Method) con los puntajes LOF

Es la técnica matemática estándar cuando tienes muchas columnas y no puedes hacer un gráfico simple. Consiste en ordenar los puntajes LOF de mayor a menor y buscar el punto de quiebre drástico (el **codo**). Se puede hacer con un gráfico de línea. Cuando la curva sube drásticamente se cuentan cuántos LOF hay en esa zona y se define esa cantidad como porcentaje del total







Reglas prácticas para fijar contamination

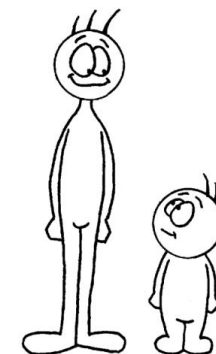
Estrategia 3: El estándar de la industria (La regla del 1% al 5%)

0.01 (1%): Se usa para escenarios donde las anomalías son extremadamente raras y costosas de revisar (ej. fraudes bancarios con tarjetas de crédito o fallas críticas en motores de avión). Solo alertamos lo que sea indudablemente un peligro

0.05 (5%): Es el estándar por defecto en la mayoría de los análisis científicos y de ciencia de datos. Asume que siempre hay un pequeño porcentaje de ruido o errores de digitación en cualquier base de datos real

0.10 (10%): Se usa en limpieza de datos (Data Cleaning) agresiva. Si vas a entrenar un modelo de Inteligencia Artificial y necesitas que tus datos estén sumamente **limpios y libres de ruido** antes de empezar, subes la contaminación para remover cualquier dato dudoso





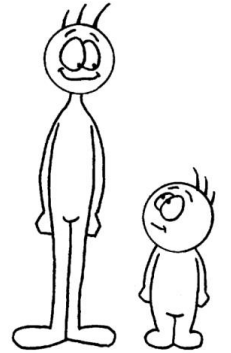
¿Cómo uso LOF para detectar outliers contextuales?

Analizaremos a 6 atletas en un gimnasio midiendo dos variables:

Frecuencia Cardíaca (BPM), con rango normal general entre 60 (si están en reposo) y 180 (esfuerzo físico)

Presión Arterial Sistólica(max), con rango normal general entre 110 (si están en reposo) y 160 (esfuerzo físico)

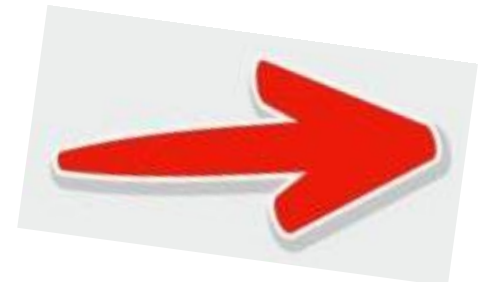
	BPM	Presión Sistólica
0	60	115
1	62	112
2	170	155
3	168	152
4	172	158
5	60	165

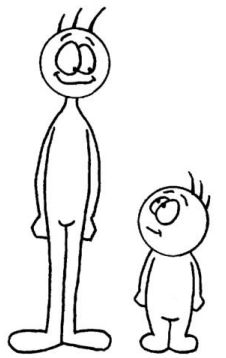


¿Cómo uso LOF para detectar outliers contextuales?

```
import numpy as np
import pandas as pd
from sklearn.neighbors import LocalOutlierFactor
#las columnas tienen magnitudes muy diferentesw
from sklearn.preprocessing import StandardScaler
datosAtletas = np.array([
    [60, 115],    # Paciente 0: Reposo (Normal)
    [62, 112],    # Paciente 1: Reposo (Normal)
    [170, 155],   # Paciente 2: Corriendo (Normal)
    [168, 152],   # Paciente 3: Corriendo (Normal)
    [172, 158],   # Paciente 4: Corriendo (Normal)
    [60, 165]     # Paciente 5: ¡Outlier Contextual! (Pulso bajo presión al límite)
])

# Escalamos para que el algoritmo mida distancias correctamente
escalador = StandardScaler()
datosEscalados = escalador.fit_transform(datosAtletas)
```



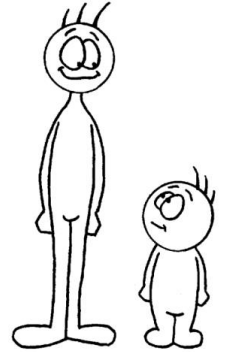


¿Cómo uso LOF para detectar outliers contextuales?

```
# Configuramos LOF (n_neighbors=2, contaminación del 16%)
modelo = LocalOutlierFactor(n_neighbors=2, contamination=0.16)
predicciones = modelo.fit_predict(datosEscalados)
puntajesLOF = -modelo.negative_outlier_factor_

# Mostramos resultados
tabla = pd.DataFrame(datosAtletas, columns=['BPM', 'Presion Sistólica'])
tabla['Predicción'] = predicciones
tabla['Puntaje LOF'] = puntajesLOF
```

	BPM	Presion_Sistolica	Prediccion	Puntaje_LOF
0	60	115	1	1.089444
1	62	112	1	1.089444
2	170	155	1	1.333333
3	168	152	1	0.875000
4	172	158	1	0.875000
5	60	165	-1	8.226253



Identificación de Datos Atípicos:

Con respecto a los outliers no hay muchas opciones en cuanto a su tratamiento

- ✓ Se ignoran los datos correspondientes
- ✓ Se emplean algoritmos de ML robustos para estos casos

El tema central acá es capturarlos y no permitir que se cuelen como situaciones normales distorsionando modelos y resultados